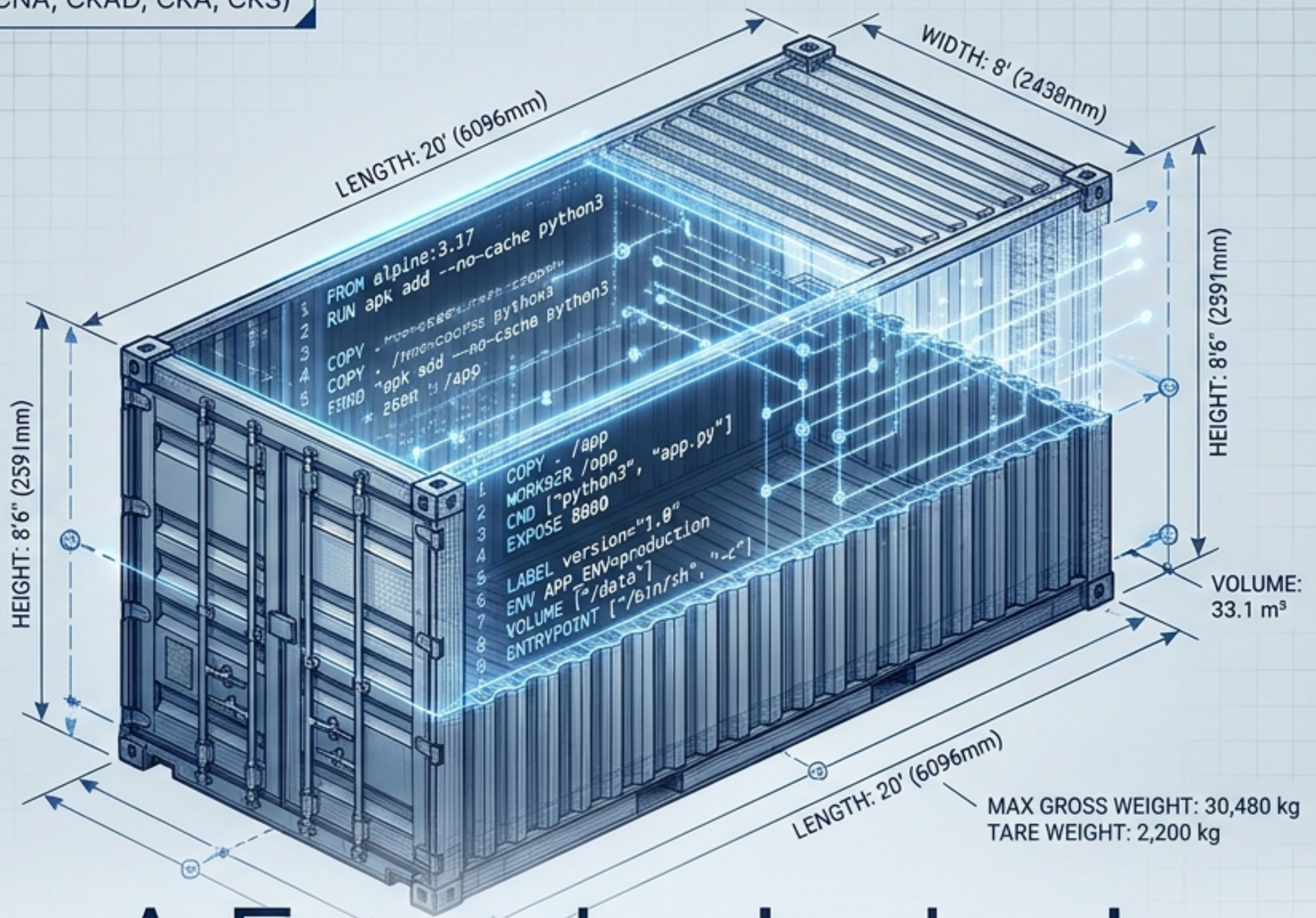
Módulo 9 | Certificação Base (KCNA, CKAD, CKA, CKS)



Docker: A Engenharia das Imagens

Do Dockerfile perfeito à execução impecável no Kubernetes.

Gemini Notebook

A Receita



Arquivo de texto versionado no Git contendo os passos (Dockerfile).

A Cozinha



O Motor (docker build). Executa a receita passo a passo no BuildKit.

O Bolo na Forma



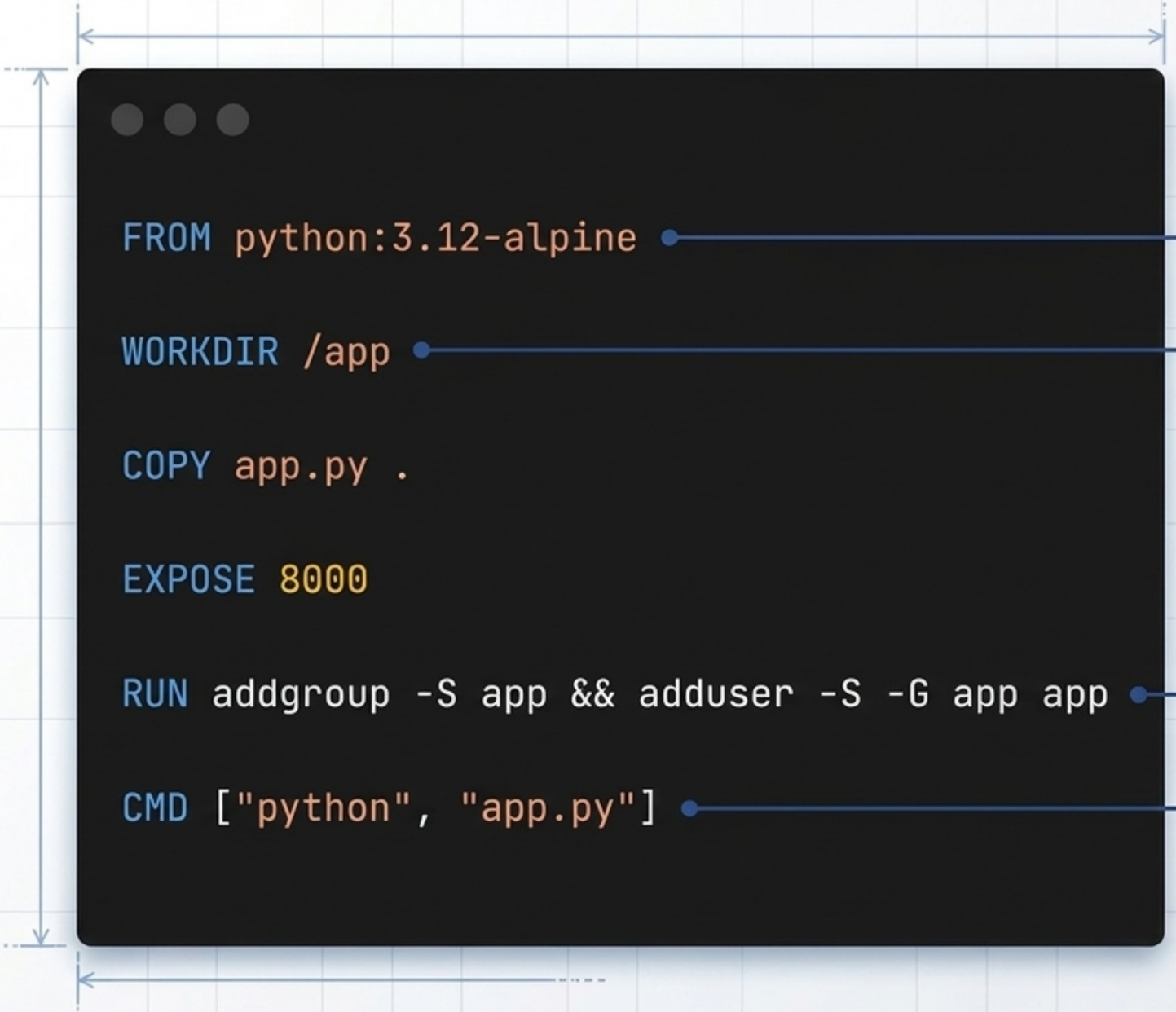
A Imagem. Um bloco estático e imutável de camadas somente leitura.

A Fatia Servida



O Contêiner. A imagem viva, em execução, coroada com uma fina camada temporária de escrita.

Gemini Notebook



A Fundação

Toda imagem nasce de outra base.

O Contexto

Define e cria a pasta de trabalho futura.

Em Tempo de Build

Acontece apenas uma vez na cozinha (ex: compilar pacotes, criar usuários).

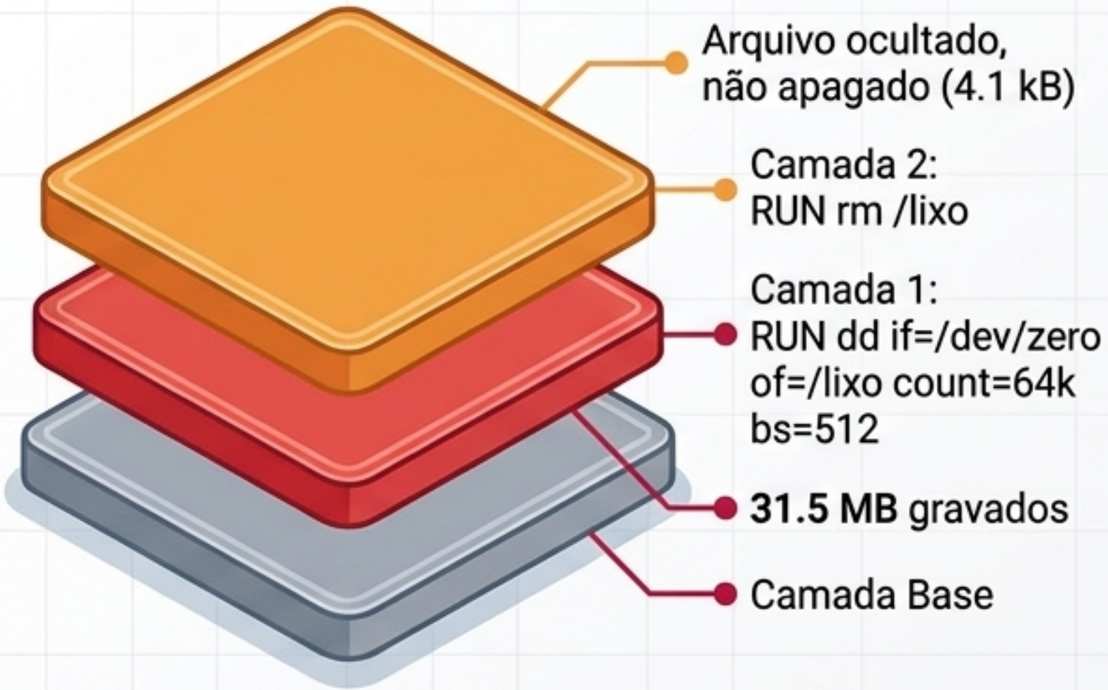
Em Tempo de Execução

Acontece toda vez que um contêiner nasce. É a identidade padrão do processo.

Gemini Notebook

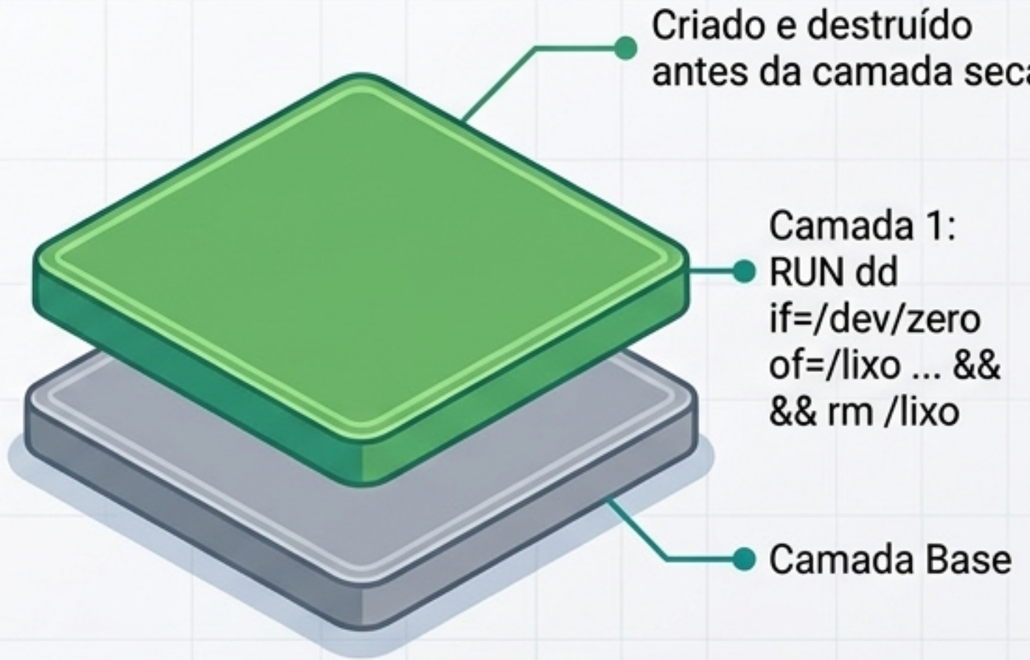
A Física das Camadas e a Armadilha do comando rm

Cenário Ruim



Resultado: A imagem final pesa **75 MB**. O lixo continua preso na camada 1.

Cenário Bom



Resultado: A imagem final pesa apenas **12.1 MB**.

⚠ **Regra de Ouro:** Encadeie comandos temporários no mesmo RUN (usando &&) para limpar o lixo antes da camada fechar. Camadas inferiores são imutáveis.

Gemini Notebook

O Motor de Cache (A Ordem Importa!)

A Ordem Ruim - Rota Vermelha

```
COPY ..
```

Copia todo o código.
Muda a cada commit.

```
RUN pip install
-r requirements.txt
```

Refeito do zero.
Download total.

Mudar uma vírgula no código **invalida o cache do pip!**

A Ordem Boa - Rota Verde

CACHED

```
COPY requirements.txt.
```

Raramente muda.

CACHED

```
RUN pip install...
```

Protegido. Usa o cache do BuildKit.

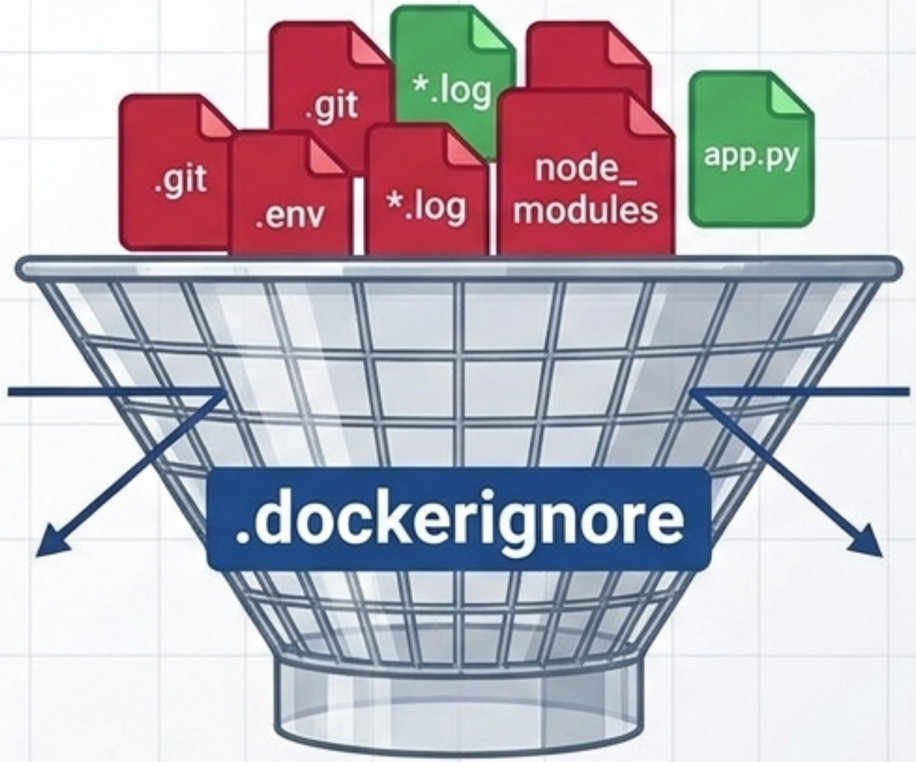
```
COPY app.py .
```

Apenas o código é refeito.

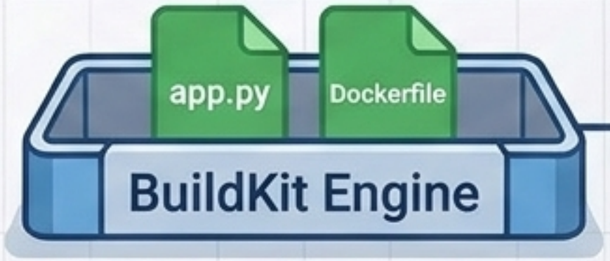
Sempre ordene de dependências (mudam menos) para o código (muda mais).

Gemini Notebook

O Filtro de Contexto (.dockerignore)



Contexto enviado:
52.44 MB



Contexto filtrado:
104 bytes

⚠ **Insight:** Sem este filtro, o comando `COPY . .` arrasta lixo, lentidão extrema e vaza segredos (senhas no .env) para dentro da sua imagem.

Gemini Notebook

A Matriz de Identidade: ENTRYPOINT vs. CMD

ENTRYPOINT
O Programa
(A Identidade, ex: python)

+

CMD
Os Argumentos
(O Padrão, ex: app.py)

=

Comando Final Executado

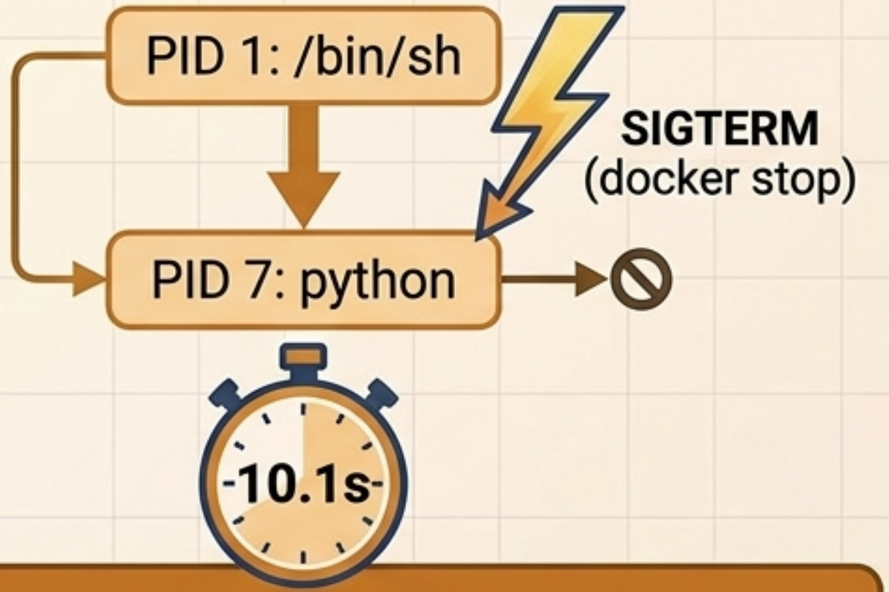
Comando de Execução	Resultado Matemático
`docker run app`	Executa: `python app.py`
`docker run app worker.py`	Executa: `python worker.py` (O argumento de terminal substitui apenas o CMD)
`docker run --entrypoint bash app`	Executa: `bash` ➔ (A **flag** substitui o programa inteiro)

Gemini Notebook

O Problema do PID 1: Shell vs. Exec

Forma Shell - Laranja

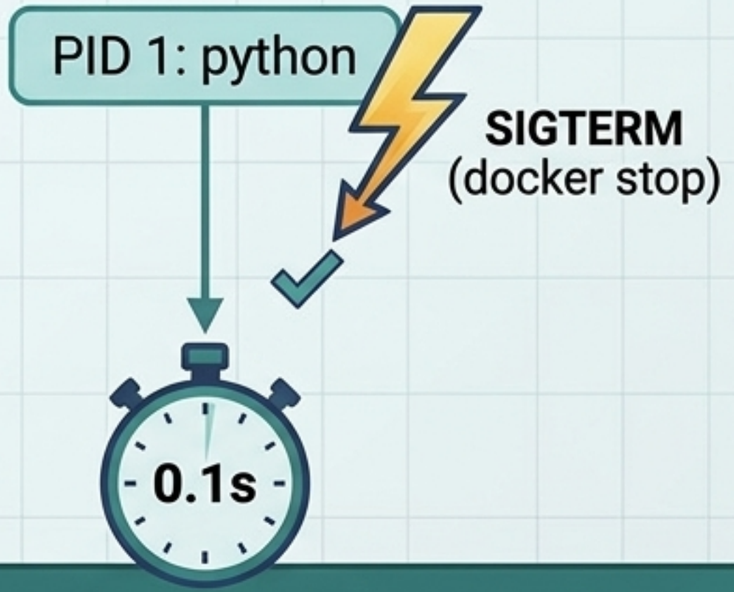
Sintaxe: `CMD python app.py` (Texto livre)



O Docker espera 10s e manda SIGKILL (Exit 137). Lento e sujo.

Forma Exec - Verde

Sintaxe: `CMD ["python", "app.py"]` (Lista JSON)



Aplicação salva o estado e desliga graciosamente (Exit 0).

⚠ **Alerta de Arquitetura:** Use sempre listas JSON. Quem não é PID 1 morre em silêncio.

Gemini Notebook

A Magia do Multi-stage Build

A Cozinha de Compilação

Etapa 1: Build

```
FROM golang:1.23-alpine AS build
```

- Compilador, bibliotecas, ferramentas, código fonte.



477 MB

```
COPY --from=build /ola /ola
```

Apenas o binário estático passa

A Entrega Final

Etapa 2: Imagem Final

```
FROM scratch
```

- O vazio absoluto.
- Apenas a aplicação.

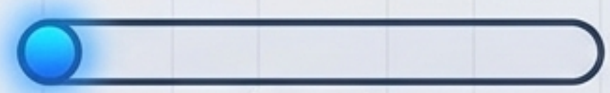


11.8 MB

Benefício de Segurança: Sem shell (/bin/sh), sem utilitários. Hackers não têm ferramentas para se mover se invadirem o contêiner.

Gemini Notebook

O Cardápio de Imagens Base

Base	Tamanho Relativo	Caso de Uso	Cuidados
Debian / Ubuntu	 (Giant)	Uso geral. Requer muitas ferramentas de sistema.	Altíssima superfície de ataque e histórico de vulnerabilidades (CVEs).
`-slim` (ex: python:slim)	 (Medium)	O melhor equilíbrio. Possui glibc e compatibilidade.	Ainda contém utilitários de shell que exigem atenção.
`-alpine`	 (Small)	Imagens mínimas, download super rápido.	Usa musl em vez de glibc (pode exigir compilação extra em C/C++).
`Scratch`	 (Zero)	O nada. Apenas para binários estáticos (Go, Rust).	Zero utilitários para debug interno. Execução às cegas.



Regra Inquebrável: Nunca use a tag `:latest`. Fixe a versão exata ou o digest SHA256 para garantir reprodutibilidade.

Gemini Notebook

Variáveis e o Perigo dos Segredos

O Cadeado Quebrado



```
ARG TOKEN=abc123
# or
ENV SENHA=xyz
```

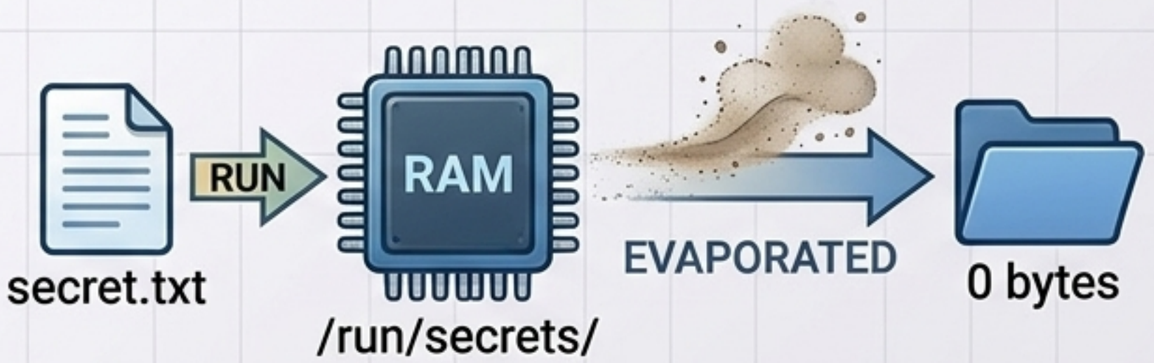
```
$ docker history --no-trunc
...
... CMD ["/bin/sh" "-c" "/app/start.sh"]
... ARG TOKEN=abc123
... ENV SENHA=xyz
...
```

Gravado para sempre. A senha fica exposta nas camadas da imagem em texto plano.

O Cofre Efêmero




```
RUN --mount=type=secret,id=tok...
```



A Solução. O arquivo é montado apenas em tempo de execução daquele RUN e desaparece sem deixar rastro (0 bytes na camada final).

Gemini Notebook

Boas Práticas: Preparando para a Produção

Identidade Segura




```
USER 10001
```

Jamais rode como root. Use um UID numérico em vez de um nome textual (**app**). O ambernesu: ID. O Kubernetes exige formato numérico para validar a regra **runAsNonRoot: true**.

Pulsação Vital



```
HEALTHCHECK
  CMD wget -qO-
  http://127.0.0.1:8000/
```

Teste a saúde interna da aplicação. Armadilha: Usar 'localhost' pode falhar em em ambientes IPv6 (resolve para ::1). Fixe sempre em 127.0.0.1.

Inspeção Contínua



```
Tool: "Linter: Hadolint"
```

Passe o **Hadolint** no seu Dockerfile para auditar automaticamente o uso de : **latest**, usuários root e pacotes obsoletos antes mesmo do build começar.

Gemini Notebook

A Grande Síntese: Do Dockerfile para o Kubernetes

Docker

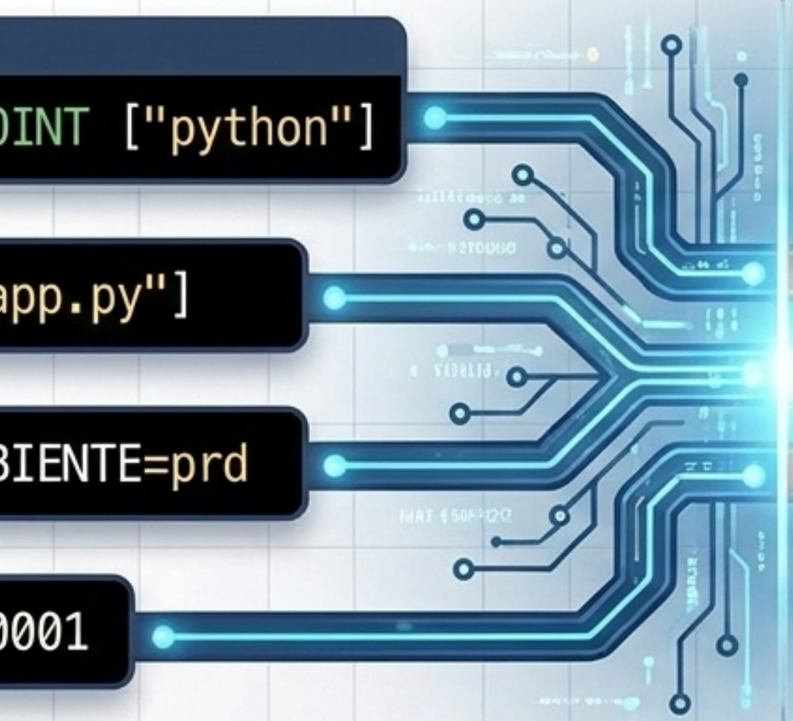
Dockerfile Block

```
1. ENTRYPOINT ["python"]
2. CMD ["app.py"]
3. ENV AMBIENTE=prd
4. USER 10001
```

Kubernetes

Kubernetes Pod YAML Block

```
A. command: ["python"]
B. args: ["app.py"]
C. env: [...]
D. securityContext:
     runAsUser: 10001
```



Nota de Arquitetura: O HEALTHCHECK definido no Dockerfile é sumariamente ignorado pelo K8s! No cluster, você deverá recriá-lo como livenessProbe e readinessProbe.

Gemini Notebook

Cheat Sheet: Inspeção e Depuração

```
1 Construir & Limpar
$ docker build -t app:1.0 .          (Build padrão com o contexto da pasta atual)
$ docker build --no-cache .          (Forçar a quebra do cache)
$ docker builder prune -f            (Limpar totalmente os resíduos do BuildKit)

2 Inspecionar a Anatomia
$ docker history app:1.0             (Ver todas as camadas, tamanhos e vazamentos de ARG)
$ docker image inspect app:1.0       (Ver labels, detalhes de entrypoint e usuário ativo)
$ dive app:1.0                       (Auditoria avançada de arquivos inúteis por camada)

3 Movimentar (Sem Registry)
$ docker save app:1.0 -o app.tar     (Exporta a imagem com todas as camadas para disco)
$ docker load -i app.tar             (Importa a imagem preservando sua arquitetura)
```

Gemini Notebook

Checklist de Sobrevivência (Ready to Deploy)

- [x] A base usa uma versão exata ou digest SHA256 em vez da tag :latest?
- [x] O arquivo .dockerignore está bloqueando ativamente segredos, logs e a pasta .git?
- [x] Os arquivos foram instalados do que “muda menos” (dependências) para o que “muda mais”?
- [x] As instruções ENTRYPOINT e CMD estão formatadas em matriz JSON **[“exec”]**?
- [x] O lixo temporário de instalação foi apagado no mesmo exato RUN que o criou?
- [x] A imagem final foi otimizada utilizando a arquitetura **Multi-stage build**?
- [x] Os segredos e senhas de build foram isolados exclusivamente com **--mount=type=secret**?
- [x] O processo principal está rodando com um UID numérico de usuário não-root (ex: 10001)?

Módulo 9 Concluído.
Próximo Passo: Docker Volumes e Redes ->

Gemini Notebook